

BDF - Customized Product Documentation

```markdown

## BDF Racer2 V3 Product Documentation

### 1. Introduction & Overview

Welcome to the BDF Racer2 V3 Product Documentation. This document provides an overview of the system's capabilities, architecture, and operational guidance tailored for BDF's bakery automation environments. Racer2 V3 is designed to enhance your production monitoring and quality control processes with reliable image acquisition and advanced AI-assisted inspection.

#### 1.1 Purpose

Racer2 V3 is the active AI Core service for line-scan acquisition, precise frame stitching, live streaming, and optional AI-assisted inspection. It is engineered to leverage two Basler line cameras, supporting both hardware-backed production and hardware-free development workflows crucial for maintaining high standards in bakery automation.

#### 1.2 Core Capabilities

Racer2 V3 offers a robust set of features to support BDF's production line:

- **Dual-Camera Acquisition:** Generates comprehensive stitched frames from two cameras for full product visibility.
- **Real-time Monitoring:** Provides browser-friendly JPEG capture and live WebSocket streaming for immediate production line feedback.
- **Flexible Camera Access:** Supports `local`, `remote`, and `emulated` modes to adapt to diverse BDF production setups.
- **Automated Quality Inspection:** Optional RF-DETR or YOLO inference for automated defect detection and product quality checks.
- **Product Color Analysis:** Tracks and buffers color data for objects, crucial for consistent product appearance.
- **External System Integration:** Facilitates `POST /analyze` for forwarding frames to external inference services.
- **Local Inference:** Enables `POST /inference` for file-driven, local AI analysis and validation.

#### 1.3 Operating Modes

The application can run in different camera and AI configurations to best suit BDF's operational needs:

**Camera Modes:**

- **`local`**: The main FastAPI service directly manages Basler cameras. Ideal for integrated setups where the server is close to the cameras.
- **`remote`**: The main FastAPI service polls a separate host-side camera service over HTTP. Suitable for distributed production lines where cameras are on a dedicated host.
- **`emulated`**: The app generates synthetic frames for debugging and UI validation without requiring physical cameras.

### AI Processing:

- **Disabled**: For raw streaming and monitoring without AI inference.
- **Enabled with `rfdetr`**: Utilizes RF-DETR for advanced detection and tracking.
- **Enabled with YOLO plus SORT**: Uses YOLO detection combined with SORT tracking by selecting a non-`rfdetr` model name.

### 1.4 High-Level Production Flow

1. Racer2 V3 starts, initiating its chosen camera runtime (local, remote, or emulated).
2. The system begins acquiring, polling, or emulating JPEG frames from the production line.
3. If automated AI inspection is enabled, model initialization runs in a background thread, ensuring monitoring can begin quickly.
4. The latest processed product frame is available immediately via `GET /capture`, `POST /analyze`, `POST /inference`, and `WebSocket /ws` for real-time monitoring and analysis.
5. When color analysis is active, tracked product samples are buffered and accessible through dedicated color endpoints for quality control.

### 1.5 Service Split for Flexibility

Racer2 V3 can operate as a single process or as two cooperating services, offering deployment flexibility for BDF:

- **Main Application**: Runs on port `8000`, handling the user interface, AI processing, and data output.
- **Optional Dedicated Camera Service**: Runs on port `8001`, managing direct access to Basler camera hardware.

This split is beneficial for BDF when the camera hardware must remain close to the production line, while the web or AI layer can be deployed separately for optimized resource allocation and network architecture.

## 2. System Architecture & Workflow

Understanding the system's architecture helps BDF operators manage and troubleshoot the Racer2 V3 effectively.

### 2.1 Runtime Layers

The system is logically divided into three layers:

1. **Entrypoints:** Launch points for the main application (`main.py`) on port `8000` and the dedicated camera service (`camera_main.py`) on port `8001`.
2. **Server Layer:** Exposes core functionalities including the main UI, streaming, AI, and color analysis endpoints (`src/server/app.py` and `src/server/camera_service.py`).
3. **Library Layer:** Contains reusable components for camera control, AI processing, and color analysis (`src/libs/racer2_camera`, `src/libs/ai`, `src/libs/color`).

## 2.2 Camera Runtime Selection for BDF Production

The main application dynamically selects its camera runtime based on BDF's deployment strategy:

- **`local`:** Directly integrates Basler camera access within the main application process.
- **`remote`:** Fetches frames and alignment data from a separate camera service, suitable for distributed BDF production lines.
- **`emulated`:** Generates synthetic frames in-process, invaluable for development and testing without physical cameras.
- **\*Note\*:** The dedicated camera service only supports `local` and `emulated` modes and does not include AI features.

## 2.3 Camera Pipeline for Production Acquisition

The live acquisition pipeline ensures high-quality image capture for BDF's products:

1. **Device Discovery:** Identifies Basler cameras via `pypylon`.
2. **Camera Requirement:** Ensures the configured number of cameras is available.
3. **Configuration:** Sets trigger, pixel format, and optional white balance for consistent image quality, unless `CAMERA_CONFIG_BY_PYLON=true` (meaning external configuration is used).
4. **Strip Acquisition:** Reads multiple strips of images from each camera.
5. **Vertical Stacking:** Stacks acquired strips vertically per camera.
6. **Offset Application:** Applies horizontal and vertical offsets for precise alignment.
7. **Image Stitching:** Combines outputs from multiple cameras into one complete product image.
8. **Color Conversion:** Converts the frame to BGR format when required by the selected pixel format.
9. **Encoding:** Resizes and JPEG-encodes the final output for efficient use.
10. **Frame Cache:** Stores the latest JPEG for immediate access by HTTP and WebSocket consumers.

- **Runtime Alignment Changes:** Operators can adjust stitching alignment via `GET /camera/alignment` (to read) and POST /camera/alignment` (to update) endpoints, allowing fine-tuning during operation.`

#### 2.4 Remote Camera Flow for Distributed BDF Setups

For BDF environments utilizing a distributed setup, `src/server/remote_camera_runtime.py` continuously polls the camera service for:`

- `/status``
- `/capture` (for frames)`
- `/camera/alignment``

This design supports deployments where the camera host maintains direct access to Basler hardware on the production line, while the main application consumes processed frames over HTTP. This allows BDF's web and AI stacks to be deployed separately for optimized resource allocation and network architecture.

### 3. AI & Data Monitoring

Racer2 V3's AI and data monitoring capabilities provide BDF with advanced tools for automated inspection and quality control of bakery products.

#### 3.1 AI Pipeline for Automated Product Inspection

The AI orchestration centralizes automated inspection for BDF products:

- **Model Selection:**
- `rfdetr`:` Utilizes RF-DETR for sophisticated detection and tracked annotation.
- Any other value: Employs YOLO detection combined with SORT tracking for general inspection and defect identification.
- **Key Behavior for Operators:**
- **Asynchronous Initialization:** AI initialization runs in the background, preventing server startup delays and allowing faster system readiness.
- **Status Monitoring:** The `/status` endpoint reports AI checkpoint progress, enabling operators to track the readiness of the AI system.`
- **Continuous Feedback:** The WebSocket stream sends raw frames until the AI is fully ready, providing uninterrupted visual monitoring.
- **Offline Analysis:** `run_single_image_inference()` supports the POST /inference` endpoint for analyzing images stored on disk.`

#### 3.2 Color Analysis for Product Consistency

The AI pipeline incorporates a `ColorBuffer` to store historical data of tracked objects, a critical feature for BDF's product consistency monitoring:`

- **Per-Object Sample State:** Provides the latest color samples for individual products.
- **Rolling History:** Maintains a history of color samples for trend analysis.
- **CSV Export:** Allows export of color data in CSV format for detailed reporting and auditing.

- **Product-Shaped Output:** Generates color-analysis output structured for easy consumption by downstream quality control systems.

### 3.3 Data Consumers: Accessing Production Insights

BDF operators and integrated systems can access the latest processed frames, AI annotations, and color analysis data through various interfaces:

- **`GET /capture`:** Retrieves the latest JPEG frame from the shared cache.
- **`POST /analyze`:** Forwards the latest frame to an external inference service for additional analysis.
- **`POST /inference`:** Initiates local AI inference for a specified image file on disk.
- **`WebSocket /ws`:** Provides a real-time stream of either raw or AI-annotated JPEG frames for live monitoring.
- **Browser UI:** The frontend (`src/front/index.html`) offers a visual dashboard for operators to view frames and status.
- **Color Data Endpoints:**
  - **`GET /color-data`:** Returns the current color analysis buffer (and clears it).
  - **`POST /color-analysis`:** Returns product-shaped color analysis output (and clears the buffer).
  - **`GET /export\_color\_csv`:** Exports the current color buffer as a CSV file.
  - **`GET /delete\_all\_color\_buffer`:** Clears all buffered color analysis samples.
  - **`GET /delete\_color\_buffer/{obj\_id}`:** Clears buffered samples for a specific tracked object ID.

### 3.4 Failure Handling for Continuous Production

Racer2 V3 is designed to be resilient and soft-failing, ensuring maximum uptime and diagnostic capability for BDF's production environment:

- **Startup Tolerance:** The system continues to start even if cameras are temporarily unavailable, allowing for diagnostics.
- **Status Availability:** The `/status` endpoint remains accessible even if frame capture fails, providing insight into the system's health.
- **Structured Errors:** `/capture`, `/analyze`, and `/inference` return structured error messages when issues occur, aiding quick problem identification.
- **Operator Awareness:** WebSocket clients receive informative text statuses (e.g., `warming_up` or camera error messages) instead of crashes, keeping operators informed of system state.

## 4. Configuration & Troubleshooting

This section provides essential information for BDF operators and administrators to configure Racer2 V3 and address common operational issues.

### 4.1 Configuration for BDF Production Environments

Racer2 V3 uses a flexible configuration system to adapt to various BDF production setups.

**Configuration Sources:**

- **Runtime Settings:** Non-secret settings are stored in ``settings.json``.
- **Secrets & Environment:** Sensitive information and environment selections are in ``env`` (and optionally ``env.debug``).

**Loading Order:** Environment variable values take precedence over ``settings.json``.

- Secrets typically reside in ``env``.
- Profile-specific non-secrets are usually in ``settings.json``.
- Environment variables can override both for dynamic adjustments during deployment.

#### Key ``settings.json`` Parameters for BDF:

- **Camera Runtime:**
  - ``CAMERA_MODE``: Select ``local``, ``remote``, or ``emulated`` based on your camera setup.
  - ``CAMERA_WHITE_BALANCE``: Enable one-shot white balance for consistent product color.
  - ``CAMERA_PIXEL_FORMAT``: Defines how camera buffers are interpreted (e.g., ``BGR8``, ``YCbCr422_8``) for correct color representation.
  - ``CAMERA_CONFIG_BY_PYLON``: Set to ``true`` if camera features are configured externally (e.g., in Pylon Viewer), preventing the app from overwriting them.
  - ``REMOTE_CAMERA_BASE_URL``: The URL of the separate camera service when ``CAMERA_MODE`` is ``remote``.
  - ``EMULATED_CAMERA_IMAGE_PATH``: An optional path to a source image for the ``emulated`` camera mode, providing deterministic frames for testing.
- **AI Runtime:**
  - ``DETECTION_AND_TRACK``: Boolean to enable or disable combined detection and tracking.
  - ``AI_MODEL_NAME``: Specifies the AI model to use (e.g., ``rfdetr`` for specific inspection tasks).
- **Secret-Bearing Keys in ``env``:**
  - ``ROBOFLOW_API_KEY``: Required for Roboflow-backed AI models.
  - ``RFDETR_MODEL_ID``: Specifies the RF-DETR model ID.
  - ``INFERENCE_API_KEY``, ``INFERENCE_API_HEADER``: For external inference services.

## 4.2 Troubleshooting Common Production Issues for BDF Operators

This guide helps BDF operators quickly diagnose and resolve typical issues with Racer2 V3.

### Server Starts But No Frames Arrive:

- **Check ``/status``:** Is ``camera.ready`` ``false``? Is the WebSocket receiving ``warming_up`` messages?
- **Local Mode:** Verify ``pylon`` is installed and both Basler cameras are connected and accessible.
- **Remote Mode:** Verify ``REMOTE_CAMERA_BASE_URL`` is correct and the host-side camera service is running, confirming ``/capture`` on the camera host returns a JPEG.
- **Emulated Mode:** Confirm ``CAMERA_MODE=emulated`` and ``EMULATED_CAMERA_IMAGE_PATH`` points to a readable file if specified.

### Camera Busy Or Unavailable:

- **Close other applications:** Ensure Pylon Viewer or other processes are not using the cameras.
- **Check for old instances:** Confirm no older copy of Racer2 V3 is still running.
- **Restart host:** If driver state seems stuck, restart the host machine.

#### **Remote Camera Mode Does Not Refresh:**

- **Verify URL:** Check `REMOTE_CAMERA_BASE_URL` for typos.
- **Port Exposure:** Ensure the camera host exposes port `8001`.
- **Network Check:** Verify `GET <REMOTE_CAMERA_BASE_URL>/status` returns `200` and the main app can resolve the camera host.

#### **WebSocket Stream Shows Text Instead Of Images:**

- This is expected when the camera is warming up, initialization failed, or remote mode hasn't received a valid frame yet. The text provides status updates.

#### **AI Never Becomes Ready:**

- **Check `/status`:** Review AI initialization phase.
- **Verify Credentials:** Ensure `.env` contains valid AI credentials (e.g., `ROBOFLOW_API_KEY`).
- **Model Selection:** Confirm `AI_MODEL_NAME` is intentional and supported.

#### **`POST /inference` Returns 404:**

- **Verify Paths:** Ensure `SrcDir` points to an existing directory on the main app host and `ImageName` is correct. The combined path `SrcDir/ImageName` must exist.

#### **Color Endpoints Return Empty Data:**

- **Enable Tracking:** Confirm `DETECTION_AND_TRACK=true`.
- **AI Readiness:** Ensure the AI model is ready.
- **Buffer State:** Check if tracked objects have been processed since the last buffer clear, or if a previous call already cleared the buffer.

### **4.3 General Diagnostic Sequence for BDF Operators**

Follow these steps for a quick and effective diagnosis of Racer2 V3 operational issues:

1. Call `GET /status` to check overall system health.
2. Confirm `camera.ready` is `true` in the status response.
3. Confirm `GET /capture` works and returns a JPEG image.
4. Confirm the browser page (`GET /`) updates with live frames.
5. If AI is enabled, wait for `ai.phase=ready` in the `/status` output.
6. Once AI is ready, then test `POST /analyze`, `POST /inference`, or WebSocket streaming (`/ws`).

